SOFTWARE ENGINEERING · CHAPTER 4 SUPPLEMENT

Elicitation Artifacts: Worked Examples

Interview notes, workshop minutes, field notes, and the running requirements log they feed — Timetabling System case study

Companion to Chapter 4, Section 4.4.3. Part 1 works through the three elicitation techniques and the raw register they produce. Part 2 then picks up the entries the main chapter didn't have room for — E-003 through E-008 — and carries them through Analysis, Specification, and Validation, the same way E-001 and E-002 were carried through the main deck.

OVERVIEW

From Three Techniques to One Running Log

Interviews, workshops, and observation are different ways of gathering the same kind of raw material. Whichever technique captures a statement, it is logged the same way — as one more row in a single running Elicitation Register:

1. Interview Notes
One-on-one, semi-structured

›

2. Workshop Minutes
Group JAD session

›

3. Field Notes
Direct observation

↓ *all feed* ↓

The Elicitation Register

The next slides show one worked example of each technique, then the register entries they produced. Part 2 of this deck then follows those entries onward — through Analysis, Specification, and Validation.

TECHNIQUE 1 · INTERVIEW

Sample Interview Notes

INTERVIEWEE	INTERVIEWER	DATE · DURATION
Dr. Li — Lecturer	**Sarah Ong — Business Analyst**	**4 Feb · 25 min**

- Dr. Li teaches 3 sections/week across 2 buildings; estimates 1–2 unplanned room conflicts per month.
- Raw quote (verbatim): “When I need to swap a room urgently, right now I have to email the admin and hope she sees it in time.”
- Follow-up: asked how she'd know a swap was approved — “I just show up and hope for the best.” No confirmation step exists today.
- Wants a same-day, mobile-friendly way to check room availability before requesting a swap.

Logged as E-002. *Follow-up needed: confirm the typical notice period lecturers can give (same-day vs. 24 hr).*

TECHNIQUE 2 · WORKSHOP / JAD SESSION

Sample Workshop Minutes

SESSION	DATE · DURATION	FACILITATOR	ATTENDEES
Timetabling Requirements JAD	**10 Feb · 90 min**	**Sarah Ong**	**Dr. Li, Mr. Daniel, Student Rep, Ms. Lee (observing)**

RAW STATEMENTS CAPTURED (ATTRIBUTED)

- Mr. Daniel (Registrar's Office): "We need at least 48 hours' notice before any recurring class time is changed, or we can't notify enrolled students in time."
- Dr. Li (Lecturer): "If two lecturers request the same room at the same time, whoever submitted first should get it — but I want to see a waitlist, not just a rejection."
- Student Rep: "Group project bookings for shared study rooms should work the same way as class swaps."

Logged as E-004, E-005, E-006. *Action items: Ms. Lee to share her approval checklist; Mr. Daniel to send the 48-hr policy document.*

TECHNIQUE 3 · OBSERVATION

Sample Field Notes

SUBJECT OBSERVED	OBSERVER	DATE · DURATION
Ms. Lee — Facility Administrator	**Sarah Ong**	**12 Feb · 09:00–11:00**

- 09:02 — Opens shared inbox; 14 pending swap requests, oldest is 3 days old.
- 09:15 — Manually cross-checks each request against a paper master timetable before approving; takes 10–15 min per request.
- 09:40 — Rejects a request because the room's projector is broken — checks this against a personal spreadsheet of “problem rooms” that isn't in any official system.
- 10:20 — Approves a swap by replying to the original email; no system record is created.

Logged as E-007, E-008. *Tacit-knowledge pain point — the “problem rooms” list lives nowhere official (see Chapter 4, Section 4.4.4).*

The Running Requirements Log (Part 1 of 2)

ID	Raw Statement	Source	Date	Status
E-001	“I want to see my class schedule on my phone, not just on the desktop portal.”	Sarah (Student), interview	3 Feb	Draft
E-002	“When I need to swap a room urgently, right now I have to email the admin and hope she sees it in time.”	Dr. Li (Lecturer), interview	4 Feb	Draft
E-003	“Room swaps should never bypass my sign-off — I'm accountable for room conflicts.”	Ms. Lee (Facility Admin), interview	6 Feb	Draft
E-004	“We need at least 48 hours' notice before any recurring class time is changed.”	Mr. Daniel (Registrar), workshop	10 Feb	Draft

The Running Requirements Log (Part 2 of 2)

ID	Raw Statement	Source	Date	Status
E-005	“Whoever submitted first should get the room — but I want to see a waitlist, not just a rejection.”	Dr. Li (Lecturer), workshop	10 Feb	Draft
E-006	“Group project bookings for shared study rooms should work the same way as class swaps.”	Student Rep, workshop	10 Feb	Draft
E-007	Ms. Lee manually cross-checks every swap request against a paper master timetable (10–15 min/request).	Observation, field notes	12 Feb	Draft
E-008	Ms. Lee keeps an unofficial “problem rooms” spreadsheet checked before approving swaps.	Observation, field notes	12 Feb	Draft

Part 2

Following the Register Into the SRS

E-001 and E-002 already made the full trip in the main chapter deck. Here, E-003 through E-008 make the same trip — Analysis, Specification, then Validation.

Organizing the Rest of the Register

The same three moves from Chapter 4, Section 4.5 — sort into User Requirements, tag with FURPS+, assign a viewpoint — applied to the remaining raw statements:

ID	Raw Statement	Outcome	FURPS+ · Viewpoint
E-003	“Room swaps should never bypass my sign-off — I'm accountable.”	Constraint on UR-2 (not its own UR)	Functionality · Domain
E-004	“We need 48 hours' notice before any recurring class time changes.”	UR-3	Functionality · Domain
E-005	“Whoever submits first gets it — but show a waitlist, not a rejection.”	UR-4	Functionality + Usability · Interactor

Organizing the Rest of the Register

ID	Raw Statement	Outcome	FURPS+ · Viewpoint
E-006	“Group study-room bookings should work the same way as class swaps.”	UR-5	Functionality · Interactor
E-007	Manually cross-checks each request against a paper timetable (10–15 min).	NFR3 candidate (not a UR — an inferred need)	Performance · Domain
E-008	Keeps an unofficial “problem rooms” spreadsheet, checked before approving.	UR-6	Reliability + Supportability · Domain

Two judgment calls worth noticing:

E-003 becomes a constraint on UR-2, not its own UR — exactly like the main deck's E-003. And E-007 was never spoken by anyone; it's inferred from watching Ms. Lee work, which is why it heads toward an NFR rather than a UR.

From User Requirements to System Requirements

Req ID	Requirement	Level	Notes
UR-3	Prevent a recurring class-time change from taking effect with less than 48 hours' notice.	User	Drafted from Mr. Daniel's E-004.
SR-3.1	IF a recurring class-time change is submitted <48h before the next session, THEN reject it and show the earliest allowed date.	System	Turns the policy into a testable rule.
UR-4	Let a lecturer join a waitlist for an already-reserved room, instead of being flatly rejected.	User	Drafted from Dr. Li's E-005.
SR-4.1	WHEN a requested room is unavailable, add the requester to a FIFO waitlist ordered by submission time.	System	Resolves “first come” into a concrete queue rule.

From User Requirements to System Requirements

Req ID	Requirement	Level	Notes
UR-5	Let students book a shared study room using the same request workflow as lecturer swaps.	User	Drafted from the Student Rep's E-006.
SR-5.1	Route a student's study-room booking through the same approval workflow (UC-04) used for lecturer swaps.	System	Reuses UC-04 rather than building a parallel flow.
UR-6	Track room-availability issues (e.g., broken equipment) so they're never offered as a swap option.	User	Drafted from Ms. Lee's field-observed E-008.
SR-6.1	WHILE a room is flagged “Under Maintenance,” exclude it from all swap-availability search results.	System	Turns Ms. Lee's spreadsheet into a system-visible flag.

New FR/NFR Entries, in EARS Syntax

ID	EARS Type	Requirement
FR6	Event-Driven	WHEN a student submits a shared study-room booking, the system shall route it through the swap-approval workflow (UC-04).
FR7	Unwanted Behavior	IF a recurring class-time change is submitted with less than 48 hours' notice, THEN the system shall reject it and display the earliest permissible date.
NFR3	Ubiquitous	The system shall automatically cross-check every swap request against the master timetable, eliminating the manual 10–15 minute check Ms. Lee performs today.

FR6 realizes SR-5.1, FR7 realizes SR-3.1, and NFR3 is the formal write-up of E-007's inferred efficiency need — *notice none of these three IDs traces back to a UR the way SR-x.x does; that link lives only in the RTM.*

Validating the New Requirements

Check Type	What It Catches Here	Concrete Example
Consistency	Two approved requirements that quietly contradict each other	UR-4's FIFO waitlist could let a routine request hold a room before Dr. Li's urgent swap (UR-2) is even reviewed — the two were never reconciled.
Completeness	A requirement missing an edge case or stopping condition	UR-4 never says what happens if a waitlisted lecturer's need disappears (e.g., their class is cancelled) before a room frees up.
Realism	Whether the organization can actually deliver what's specified	UR-6 assumes staff will flag every “problem room” in real time — the field notes (E-008) show that doesn't happen consistently today.

PHASE 4 · CASE STUDY CHECKPOINT

A Validation Walkthrough Catches a Conflict

During a walkthrough with Dr. Li and Ms. Lee, a reviewer places UR-2 (urgent swap) and UR-4 (FIFO waitlist) side by side and asks a simple question: *“if an urgent swap and a routine waitlist request target the same room, which one wins?”* Neither requirement says.

The gap is logged as a Validation finding, not silently patched — it goes back to Specification for a fix: SR-4.1 is revised to add an explicit exception — **“WHERE the incoming request is flagged urgent per UC-04, the system shall place it ahead of the FIFO waitlist, subject to Ms. Lee's sign-off per UR-2.”**

This is exactly why Validation is a separate phase from Specification: it's what catches a contradiction between two individually reasonable requirements before either one gets built.

HOW THESE FEED CHAPTER 4

Same Raw Material, One Chapter-Long Story

Every entry in the Elicitation Register has now made the same trip: E-001 and E-002 through the main chapter deck (→ UR-1/UR-2 → SR-1.1/SR-2.1); E-004 through E-008 through this supplement (→ UR-3 through UR-6, SR-3.1 through SR-6.1, FR6, FR7, and NFR3). Only E-003 stayed a constraint rather than becoming a UR of its own — by design, not by omission.

One phase is left: **Traceability & Change Control** — pulling every artifact from both decks into one governed document. The remaining slides do exactly that: a complete, assembled SRS for the Timetabling System, structured exactly as Section 4.6.1 teaches.

SRS

The Complete Software Requirements Specification

Every UR, SR, FR, NFR, and use case from both decks, assembled into one baselined document — following the Standard SRS Structure from Section 4.6.1.

SRS SECTION 1

Introduction

1.1 Purpose

Defines the functional and non-functional requirements for the University Timetabling System: schedule viewing, room-swap requests (routine and urgent), waitlisting, shared study-room booking, and recurring-change control.

1.2 Scope

In scope: schedule display, swap/waitlist workflows, study-room booking, change-notice enforcement, room-status flagging, and related notifications. Out of scope: payment processing, non-academic facility bookings, push-notification infrastructure (assumed provided by the platform).

1.3 Definitions, Acronyms & Glossary

See the main chapter's Glossary and ID Legend (Section 4.6.1's Glossary of Key Terms) for E-xxx, UC-xx, UR-x, SR-x.x, FRx/NFRx, and TC-xxx.

1.4 References & Document Overview

References: the Elicitation Register (this supplement) and Chapter 4, Sections 4.4–4.8. Overview: Section 2 describes the product; Section 3 lists every requirement; Section 4 holds supporting appendices; Section 5 records sign-off.

SRS SECTION 2

Overall Description

2.1 Product Perspective. A standalone mobile and web application that integrates with the university's existing Student Information System and ERP over REST APIs — it does not replace either.

Schedule Viewing

Mobile calendar view of a student's or lecturer's weekly timetable.

Room Swap & Waitlist

Request, approve, and — when unavailable — waitlist a room/time change.

Study Room Booking

Student bookings for shared study rooms, via the swap-approval workflow.

Change Control & Status

48-hour notice enforcement and “Under Maintenance” room flagging.

2.3 User Characteristics. Students (mobile-first, casual); Lecturers (frequent, time-pressured); Facility Administrators (power users, need oversight); Registrar staff (occasional, policy-focused).

2.4 Constraints. Must integrate with the existing ERP; must perform over 4G mobile networks; subject to the institution's data-privacy policy (drives NFR2).

2.5 Assumptions & Dependencies. Assumes the university's student database remains available and campus network coverage is adequate for 4G performance targets.

SRS SECTION 3.1

Functional Requirements

ID	Module	Requirement
FR1	Scheduling	The system shall display weekly class schedules in a calendar view.
FR2	Notifications	WHEN a room change request is submitted, the system shall alert the administrator.
FR3	Scheduling	WHILE database sync is in progress, the system shall show a loading indicator.
FR4	Accessibility	WHERE text-to-speech is enabled, the system shall read out room details.
FR5	Authentication	IF an invalid university ID is entered, THEN the system shall show an error.
FR6	Study Room Booking	WHEN a student submits a study-room booking, the system shall route it through UC-04.
FR7	Change Control	IF a recurring change is submitted <48h before the next session, THEN the system shall reject it.

SRS SECTION 3.2

External Interface Requirements

- User Interfaces — a mobile calendar app for students and lecturers, and a web-based administrative dashboard for facility staff and the Registrar.
- Hardware Interfaces — none beyond a standard smartphone or laptop with a modern browser; no special-purpose hardware is required.
- Software Interfaces — a REST API to the university's ERP (course codes, semester data) and to the Student Information System (enrollment records), per Section 2.1.
- Communication Interfaces — HTTPS with TLS 1.3 for all client-server traffic (realizes NFR2), plus a push-notification channel for swap-status and waitlist alerts.

SRS SECTION 3.3

Non-Functional Requirements

ID	FURPS+ Category	Requirement
NFR1	Performance	The mobile app shall load timetables within 3.0 seconds over a 4G connection.
NFR2	Security (+constraint)	The system shall encrypt schedule and swap-request data in transit (TLS 1.3) and at rest (AES-256).
NFR3	Performance / Supportability	The system shall automatically cross-check every swap request against the master timetable.

NFR2 was added via ECR six months after baselining (Section 4.8.2) — it did not exist in the original v1.0 catalog. See the Revision History in Section 4 for the full baseline timeline.

Use Case Specifications & Models

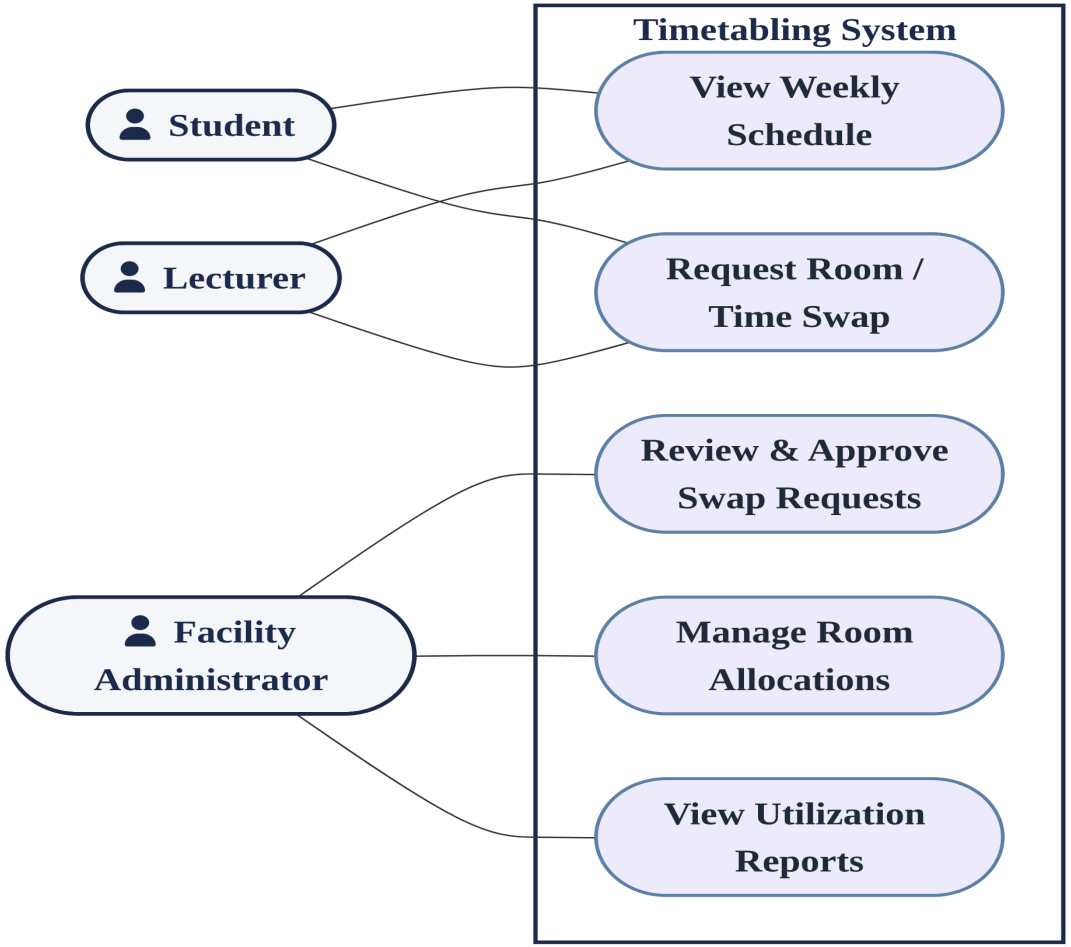


Fig. 4.1 — High-level use case diagram, Timetabling System.

- UC-04 — Request Room / Time Swap. Fully specified in the main chapter deck (extensions 5a/5b cover capacity and concurrency); FR6 and SR-5.1 reuse this same workflow for study-room bookings.
- View Weekly Schedule, Manage Room Allocations, View Utilization Reports. Drafted informally during Elicitation; finalized use case descriptions live in the project's design documentation, not reproduced here.

Requirements Traceability Matrix

Req ID	Description	Level	Design Module	Test Case	Status
UR-1	View weekly schedule on mobile.	User	TimetableUI	TC-UI-101	Approved
SR-1.1	WHEN tab opened, fetch JSON <2.0s over 4G.	System	ScheduleCtrl	TC-API-204	Passed
UR-2	Lecturer can request urgent room swap.	User	SwapModule	TC-UI-105	Approved
SR-2.1	IF capacity < enrollment, THEN reject swap.	System	CapacityChk	TC-UNIT-402	Passed
UR-3	48h notice before recurring class-time change.	User	ChangeCtrl	TC-UI-120	Approved
SR-3.1	IF <48h notice, THEN reject + show earliest date.	System	ChangeValidator	TC-UNIT-410	Passed

Requirements Traceability Matrix

Req ID	Description	Level	Design Module	Test Case	Status
UR-4	Waitlist instead of flat rejection for a busy room.	User	WaitlistModule	TC-UI-125	Approved
SR-4.1	FIFO waitlist; urgent requests jump the queue.	System	RoomWaitlistSvc	TC-API-215	Passed
UR-5	Students book shared study rooms via swap workflow.	User	StudyRoomModule	TC-UI-118	Approved
SR-5.1	Route booking through the UC-04 approval workflow.	System	StudyRoomForm	TC-UI-119	Passed
UR-6	Flag rooms with availability issues.	User	RoomStatusModule	TC-UI-130	Approved
SR-6.1	Exclude “Under Maintenance” rooms from search.	System	AvailabilityFilter	TC-UNIT-422	Passed

Data Dictionary & Revision History

Appendix B — Data Dictionary (excerpt)

- Class Session · Room · Swap Request · Waitlist Entry · Study Room Booking · Room Status Flag — the core entities referenced across FR1–FR7 and SR-1.1–SR-6.1; see the project data model for full field-level definitions.

Appendix C — Revision History

Version	Change	Approved By
v1.0	Initial baseline: UR-1–UR-6, SR-1.1–SR-6.1 (SR-4.1 already includes the urgent-request exception found during Validation), FR1–FR7, NFR1 & NFR3.	Joint Steering Committee
v1.1	Six months later: NFR2 (encryption in transit/at rest) added via Engineering Change Request.	Change Control Board

SRS SECTION 5

Approval / Sign-off

Baselined as v1.0 by the Joint Steering Committee (Section 4.6.2); this signature block records that approval.

Role	Name	Signature	Date
Lead System Architect & Business Analyst	Sarah Ong		
Client Representative / Project Sponsor	Dr. Amara Osei, Dean of Academic Affairs		
Project Manager	Rafael Cruz		

THE COMPLETE PICTURE

From a Stakeholder's Sentence to a Signed Document

Every ID in this SRS started as someone's raw sentence: Sarah wanting her schedule on her phone, Dr. Li wanting an urgent swap and, later, a waitlist, Mr. Daniel's 48-hour policy, the Student Rep's study-room idea, and Ms. Lee's unofficial “problem rooms” spreadsheet. Elicitation captured them, Analysis organized and classified them, Specification made them precise and testable, Validation caught the one contradiction that would have shipped a broken waitlist, and Traceability now ties every one of them to a design module and a test case in a single governed document.

That is what “doing Requirements Engineering” actually produces — not a slide deck, but this.